HDOC DAY – 46

ALGORITHM:

Main Algorithm:

1. Start.
2. Read the size of the array.
3. If size <= 0, display an error and stop.
4. Declare the variable-length array nums[size].
5. Read all elements into nums.
6. Call Bubble_sort(nums, size).
7. Display the sorted array.
8. Call occurrence(nums, size).
9. Stop.

Algorithm for Bubble_sort():

1. Receive the address of nums and its size.
2. Repeat for size - 1 passes.
3. Compare each pair of adjacent elements.
4. If the left element is greater than the right element, swap them.
5. Continue until the array is sorted in ascending order.
6. Return to main().

Algorithm for occurrence():

1. Receive the address of the sorted array and its size.
2. Read target from the user.
3. Initialize first = -1 and last = -1.
4. Traverse the array from the beginning.

5. When target is found, store its index in first and stop the search.
6. If first != -1, traverse the array from the end.
7. When target is found, store its index in last and stop the search.
8. Display the target value and the indices of its first and last occurrences.
9. If the target was not found, display -1, -1.

C PROGRAM:

```
#include <stdio.h>


// Function to sort the array using Bubble Sort
void Bubble_sort(int *nums, int size)
{
   int i, j, temp;


   for (i = 0; i < size - 1; i++)
   {
      for (j = 0; j < size - i - 1; j++)
      {
         if (*(nums + j) > *(nums + j + 1))
         {
            temp = *(nums + j);
            *(nums + j) = *(nums + j + 1);
            *(nums + j + 1) = temp;
         }
      }
   }
}
```

```
// Function to find first and last occurrence
void occurrence(int *nums, int size)
{
   int target;
   int first = -1, last = -1;
   int i;

   printf("Enter target: ");
   scanf("%d", &target);

   // Find first occurrence
   for (i = 0; i < size; i++)
   {
      if (*(nums + i) == target)
      {
         first = i;
         break;
      }
   }

   // If target is present, find last occurrence
   if (first != -1)
   {
      for (i = size - 1; i >= 0; i--)
      {
         if (*(nums + i) == target)
         {
            last = i;
```

```
            break;
        }
      }

      printf("First occurrence: %d at index %d\n",
          *(nums + first), first);

      printf("Last occurrence: %d at index %d\n",
          *(nums + last), last);
   }
   else
   {
      printf("-1, -1\n");
   }
}

int main()
{
   int size, i;

   printf("Enter size of array: ");
   scanf("%d", &size);

   if (size <= 0)
   {
      printf("Invalid array size.\n");
      return 0;
   }
```

```
    // Variable-length array
    int nums[size];

    printf("Enter %d elements:\n", size);

    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    // Passing address of array
    Bubble_sort(nums, size);

    printf("\nSorted array: ");

    for (i = 0; i < size; i++)
    {
        printf("%d ", nums[i]);
    }

    printf("\n");

    // Passing sorted array to occurrence()
    occurrence(nums, size);

    return 0;
}
```

TEST CASE TABLE:

Test Case	Input Array	Target	Sorted Array	Expected Output
1	10 8 7 5 8 7	8	5 7 7 8 8 10	First: 8 at index 3, Last: 8 at index 4
2	10 7 5 8 7	8	5 7 7 8 10	First: 8 at index 3, Last: 8 at index 3
3	8 5 10 7 8 7	6	5 7 7 8 8 10	-1, -1
4	5 5 5 5	5	5 5 5 5	First: 5 at index 0, Last: 5 at index 3
5	9	9	9	First: 9 at index 0, Last: 9 at index 0